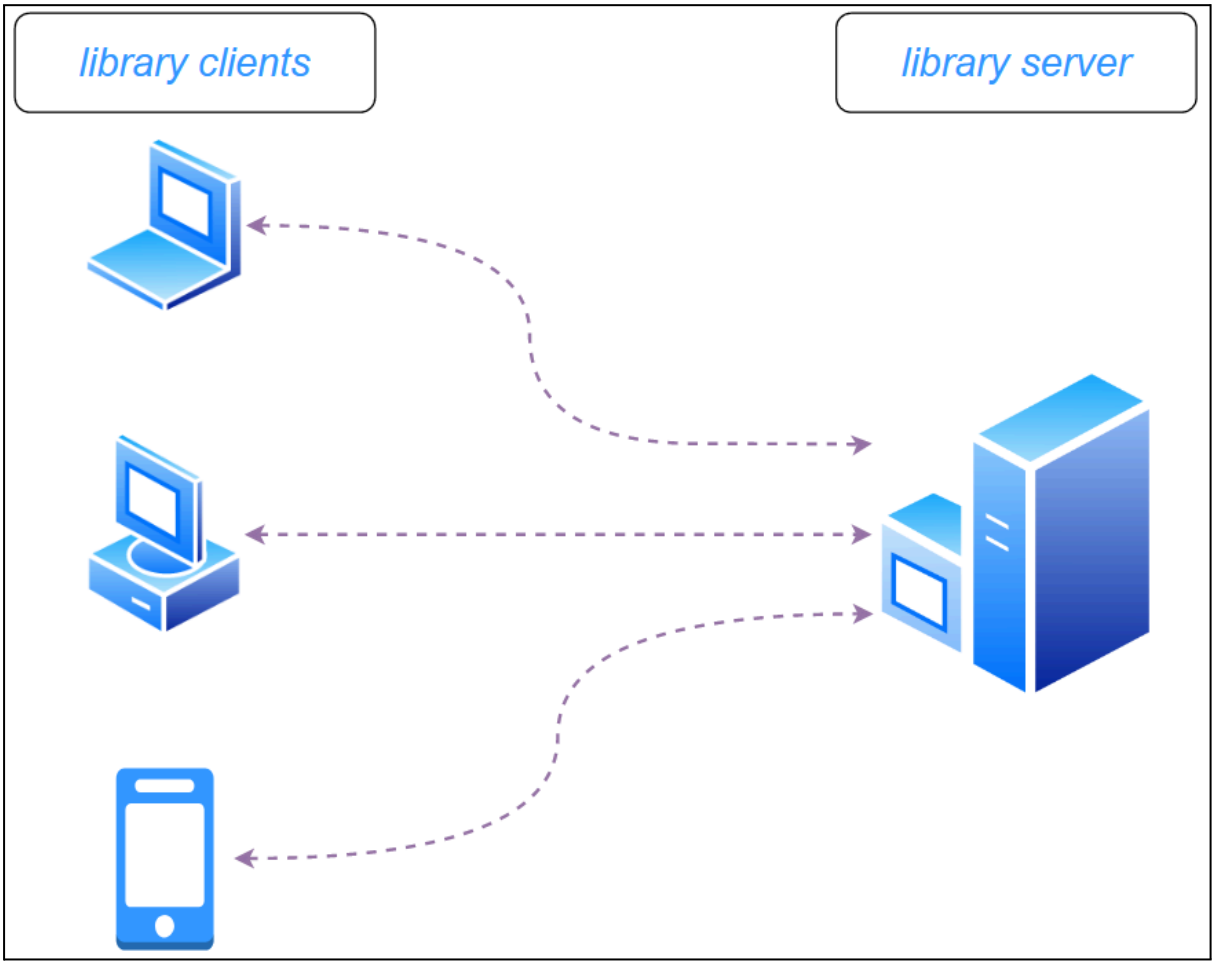
Library - Development Guide

Prof. Tornese Gianluca
11/10/2022



Introduction

This project, based on the Java programming language and the Object-Oriented methodology, is designed to introduce the concepts of socket programming and message exchange.

We will simulate a software system running a small library, loading up a catalogue of 100.000 books, hosted by a java application (*library server*).

This application will open up a *TCP* port for incoming connections of client applications (*library client*). Multiple connections are accepted simultaneously, simulating what a real server application would do.

Our library will work when both applications (*clients* and *server*) are hosted on the same computer, or on different ones, as long as there's a connection between the different hosts and there's no firewall blocking policy in place.

We will design a small message protocol, for clients to send requests and server to provide responses in a harmonized shared manner.

As for previous projects, when pseudocode is provided, it always contains some typos and inaccuracies intentionally, not to make it a plain copy-paste exercise.

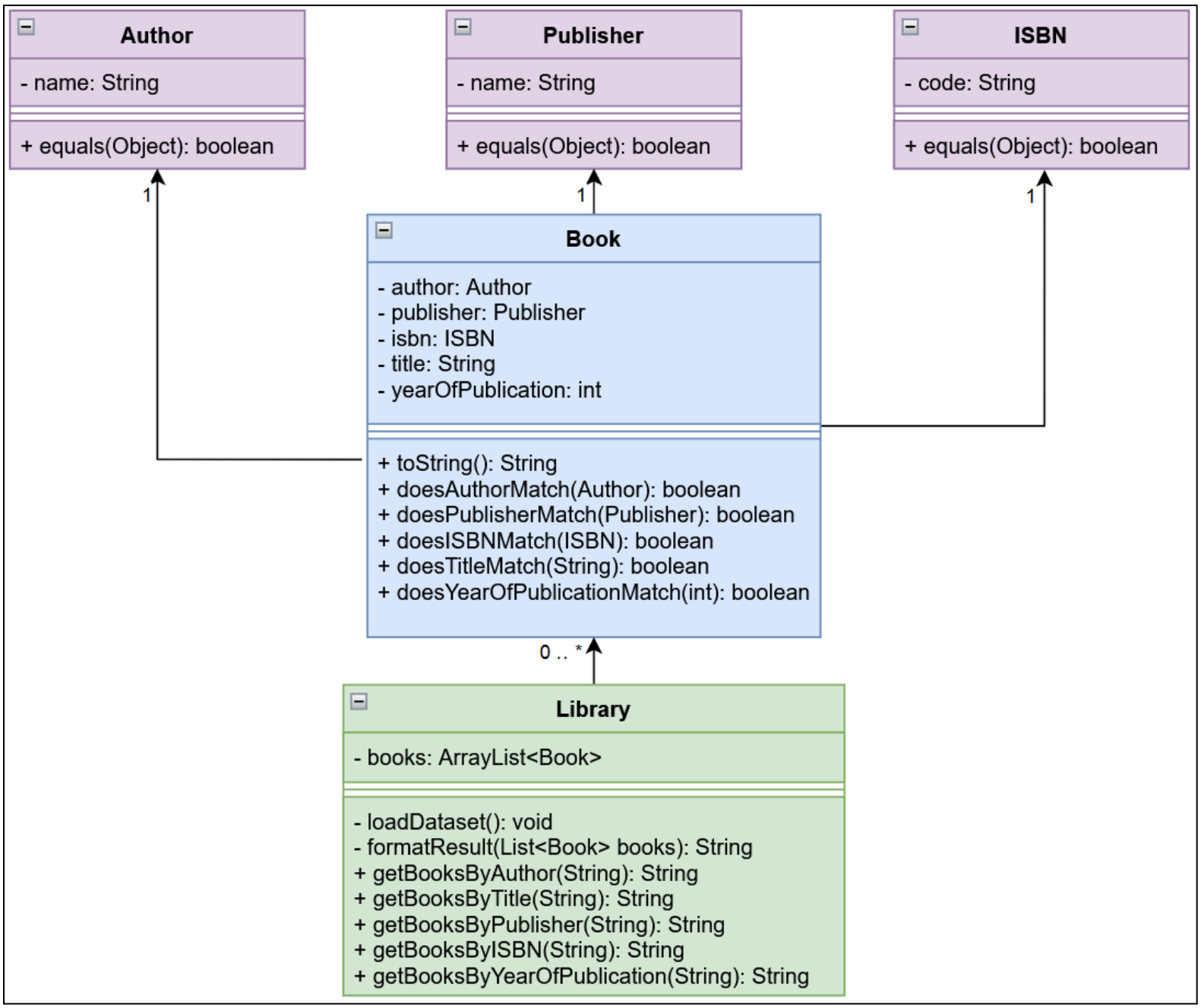
Next section will provide an overview of our development activities for both applications by showing the class diagrams.

Projects' overview

Library Server

The following *UML* class diagrams summarize the classes we will define for our server application (*library server*).

The following class diagram shows the classes of the *core* package of the *library server* project.



A *book* is defined in terms of its *author*, *title*, *isbn* code, *publisher* and *year of publication*.

The *book* class offers methods to check whether a given field matches or not (for example if two books are written by the same author).

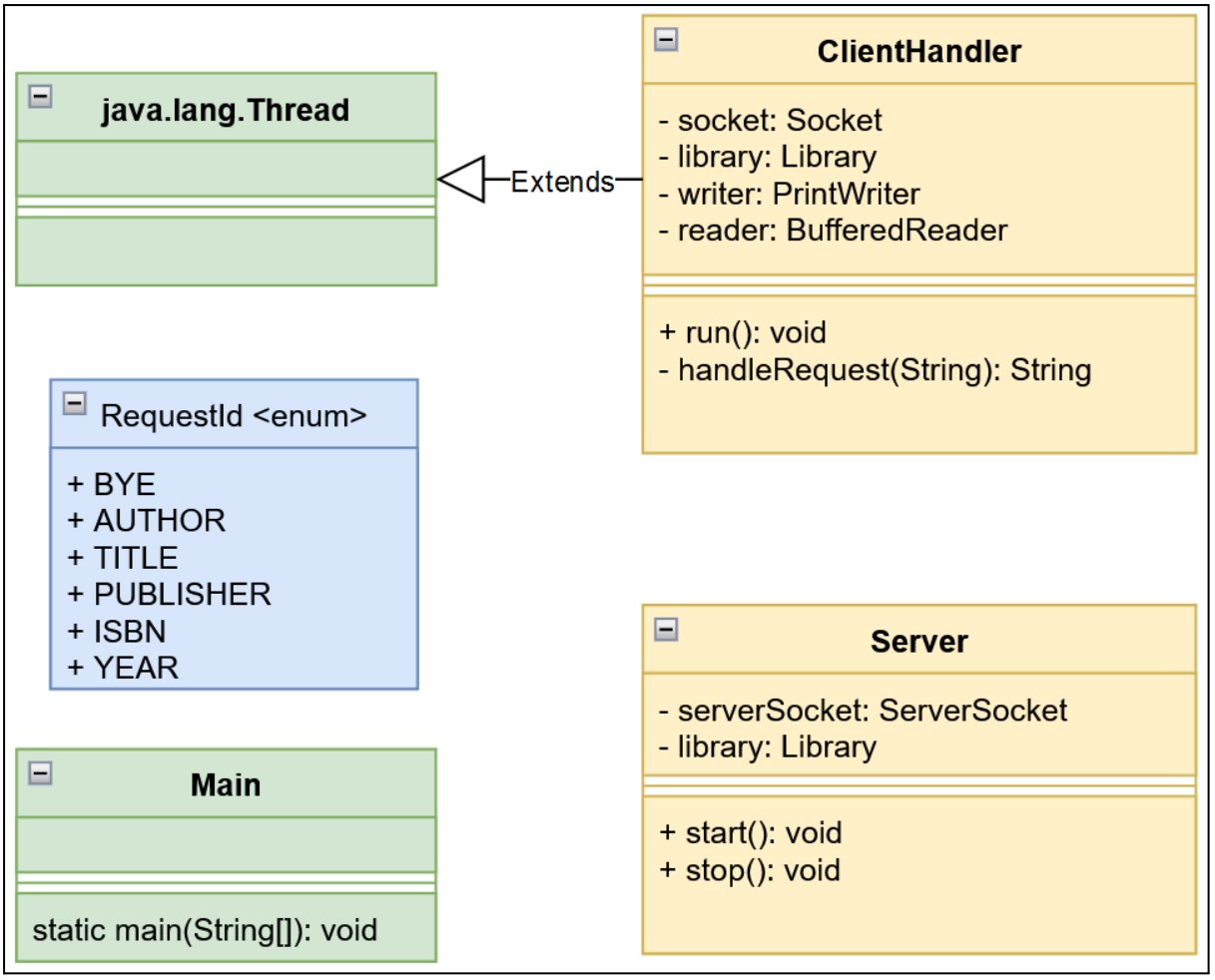
A *library* is a collection of books, and offers functionalities to load a catalogue of books from a .CSV file (explained in details later on), and to perform queries on it.

The following class diagram shows the content of the *server* package of *library server*.

We define an enumeration (*RequestId*), to ease the usage of our message protocol.

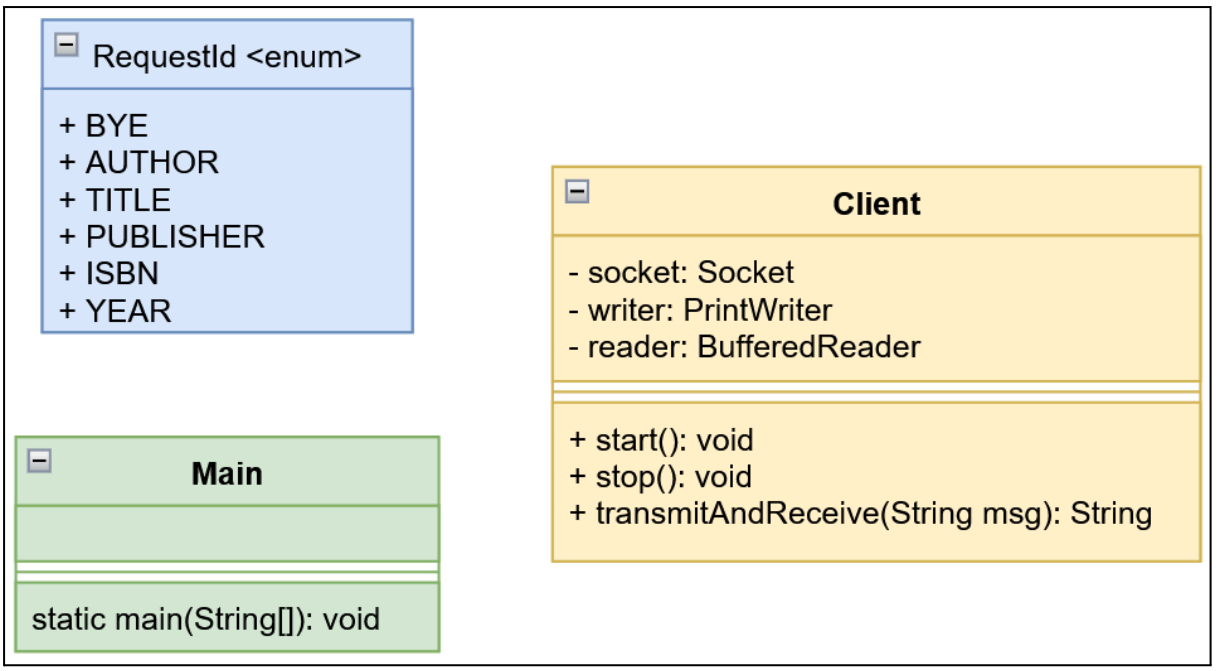
We have a *Server* class, responsible for accepting incoming connections on a given *TCP* port, and spawning *ClientHandler* instances simultaneously to handle the requests.

Finally, for the application to be launched, we will have our usual *Main* class.



Library Client

Our code gets easier on the *client* side. We will have a single package (*client*), whose content is shown in the following class diagram.



We need to have a copy of the *enumeration RequestId*, to share the message protocol definitions.

We have a *Client* class that connects to the *library server* and exercises its functionalities (query the books' catalogue).

And, as usual, our *Main* class, for the application to have a starting point.

Message protocol

We need to define a protocol over *TCP* for clients and servers to exchange messages.

Clients will make requests, and server will provide responses.

Hence, we need to define a communication protocol in both directions.

We will keep things simple, and send back and forth plain strings (*UTF-8* sequences).

Clients to Server

Clients will query the books' catalogue by either specifying:

- the author
- the title
- the *isbn* code
- the publisher
- the year of publication

Clients hence will always start messages by specifying an identifier (the *RequestId* enumeration), followed by a *fixed delimiter (" - ")*, the *actual query content*, and the next line *(\n)* delimiter.

The following message queries the server by author (id 1) *Lewis Carroll*:

> *1 - Lewis Caroll\n*

Server to Clients

Our server replies to clients' queries with all books that match the request.

The response is hence made of multiple lines (multiple books might match), and we need on clients' side to understand when a response is over (the **\n** delimiter won't be sufficient anymore).

Here is an extract of the books that matches the request made by client in the previous paragraph:

> *140620869 - Alice's Adventures in Wonderland (Penguin Popular Classics) - Lewis Carroll - Penguin Books Ltd - 1994\n*
> *9504001068 - Alicia a Traves del Espejo - Lewis Carroll - Alianza - 1993\n*
> *486408787 - Through the Looking-Glass: And What Alice Found There (Dover Thrift Editions) - Lewis Carroll - Dover Publications - 1999\n*
> *@@@\n*

Each line contains:

- the *isbn* code
- the delimiter
- the title
- the delimiter
- the author
- the delimiter
- the publisher
- the delimiter
- the year of publication
- the \n delimiter

Additionally, a last "special" line (*"@@@\n")* is appended, to mark the end of the message.

Development

Library Server

First things first, let's start by creating a java application named *libraryServer*, composed of two packages, *core* and *server*.

In the project folder, next to the *src* sub-folder, copy the .CSV archive file *books.csv*, that we will use to load our books' dataset.

core package

Let's start by developing the classes of the *core* package.

Author

Author is the first class that we are going to build. Nothing fancy, it contains a single private and final attribute, named *name* of type *String*.

We receive and initialize this attribute in the constructor, in the usual manner.

Let's override the *toString* method, that shall return the *name* attribute to represent the instance.

Also, we need to *override* the *equals* method (returns a boolean, receives an *Object*). We will need it to determine whether two authors match or not.
As usual when *overriding* the *equals* method, let's verify if the two classes match, as follows:

```
if o.getClass() differs from getClass()
        return false;
```

We have now covered the case of a distracted user that would compare our instance with an instance of another object.

Now that we know that we are comparing the right type of object, let's cast it to *Author*, and compare the only significant attribute (*name)*.

In order not to take into account capital letters, abbreviations and such, we will compare the *name* attributes converted into their lower case format, and use the *contains* method rather than *equals*.

The result shall look as follows:

```
auth = (Author) o;
return name.toLower does it contain (auth.name.toLower  )
```

And this is all for *Author*, we could say that this class is pretty much designed only to encapsulate the logic of comparison just described.

ISBN

This class is very simple and very similar to the previous one.

It only contains an attribute, being *private* and *final* and initialized in the constructor, named *code* and of type *String*.

It overrides the *toString* method, providing back *code*.

It overrides the *equals* method, checking that classes do match as in the previous paragraph, and comparing *code* attributes through the *equalsIgnoreCase* method.

Publisher

Nothing fancy again, the logic to develop is identical to what applied for the *Author* class. Time for some copy-paste action.

Book

This class relies on all previously defined ones.

It has several attributes, all *private*, *final* and initialized in the *constructor* method; these are:

- *author*
- *isbn*
- *publisher*
- *title (*of type *String*)
- yearOfPublication (of type *int*)

It overrides the *toString* method, by returning a concatenation of all attributes, for example:

> *9504001068 - Alicia a Traves del Espejo - Lewis Carroll - Alianza - 1993*

Would you say it is a coincidence that this representation matches the server response format seen in the previous paragraphs?

This concept of encoding a class into a structured representation is referred to as *serialization* (meaning one field after the other). The decoding is referred to as *deserialization*.

Now we implement the following five methods:

- doesAuthorMatch
- doesPublisherMatch
- doesISBNMatch
- doesTitleMatch
- doesYearOfPublicationMatch

Each of them is *public*, returns a *boolean* as name would suggest, and takes as parameter the proper object type (*Author, Publisher, …*).

What do they do? They simply tell us if a given attribute, let's say the *isbn* code, matches or not the ones of the given instance, by relying when possible on the underlying *overridden equals* methods.

Library

This class represents the collections of books, and provides methods to load a .CSV dataset, provided as part of the project.
Let's start by the attributes, we need a single one representing the set of books.

We will name it *books*, and pick as data structure the *ArrayList<Book>*, since this choice gives us a bit more flexibility with respect to plain arrays. As usual, we apply the *private* and *final* attributes.

The constructor takes no parameters, and shall create the *ArrayList<>*. It shall also invoke the *loadDataset* method, that comes next!

Let's now define a *loadDataset* method, that takes no parameters and returns nothing.

It loads the .CSV dataset up in memory. The .CSV is a data format that stores entries as a set of rows, in which each attribute is separated by a *comma* delimiter (CSV stands for *Comma Separated Value*).

For example *Google* and *Office Sheets* may be exported in this text format.

In this project we load the *books.csv* dataset, that appears as follows:

> *ISBN,Book-Title,Book-Author,Year-Of-Publication,Publisher*
> *195153448,Classical Mythology,Mark P. O. Morford,2002,Oxford University Press*
> *2005018,Clara Callan,Richard Bruce Wright,2001,HarperFlamingo Canada*
> *60973129,Decision in Normandy,Carlo D'Este,1991,HarperPerennial*
> *…*
> *…*
> *…*

As you can see the first row is a *header* that tells us the meaning of each column. Each row represents a book.

I've prepared and cleaned up this dataset so as not to have any exceptional row (containing weird symbols, commas that might be misinterpreted and null fields), and to offer exactly 100.000 books.

Back to the *loadDataset* function, in order we need to:

- define a *COMMA_DELIMITER* String, initialized to the "," character. Not every .csv archive uses commas as delimiters as the name would suggest, and so here we define ours.
- we define a variable named *records*, whose type is *List<List<String>>*. Let's create it in the *heap* as *new ArrayList<>*.
  You might wonder why a list of list of strings? Each list is a row, and is hence composed of a list of strings (column fields). Makes sense?
- we need a way to read the archive line by line, and this is achieved through the *BufferedReader* class. Let's create a variable of this type, passing our dataset as parameter through a *FileReader* instance:

  br = new BufferedReader(new FileReader("books.csv"));
- one more variabile before the loading action starts, a plain String named *line*.

- since we need to read a lot of lines (100000), all with the same format, a *cycle* is a must. Theoretically we do not know the lines a priori, hence we go for a *while* cycle rather than a *for*, and we:
    - check in the loop condition if the result of *BufferedReader readLine* is valid
    - if so (hence we are inside the loop) we retrieve an array of strings by applying the comma delimiter, throuh the *split* function
    - and finally we add the string array to our *records* data structure, converting it to list first

  Easier done than said, it shall appear as follows:

  ```
  while ((line = br.readLine()) is different from null) {
      String[] values = line.split(our comma delimiter);
      records add (Arrays.asList(values));
  }
  ```

  Each iteration loads up an array of strings, converts it to a list, and adds it to our list of lists.
- Now we need to iterate through the list, and get our book instances out of the strings. To spice things up, we will try using a *ListIterator* object, defined as follows:

  ```
  ListIterator<List<String>> recordsIter = records.listIterator();
  ```

  This object is meant for iterating through lists, and has a very simple syntax. We define a *while* cycle that iterates up until the iterator has a *next* element, and on each element we retrieve each *field* attribute through the *get* method.

  Before running the cycle though we need to skip one element of the dataset, being the *header* of the archive, that does not represent a book.
  Here's the pseudo code:

  ```
  recordsIterator.next // let's skip the header

  // iterate all records
  while (recordsIter.hasNext  ) {

      // each record is a set of strings, describing a book
      List<String> record = recordsIter.next

      isbn = new ISBN(record.get(0))
      title = record.get(1)
      author = …
      yearOfPublication = …
      publisher = …

      // create the book and add it to our dataset
      book = new Book(title, author, …, …, …)
      books.add(book)
  }
  ```

- Before concluding, the method shall print how many books were loaded, for you to verify that all 100.000 entries were parsed successfully. Simply print out *books.size()* with a nice explanatory message.

Let's now define a helper *private* method, called *formatResult*. It takes as input a list of books, named *books*, and returns a string.

As the name suggests, it computes a nicely formatted string representing all books received.

Do you remember that the *toString* method is invoked implicitly each time we try to print an object? So all we need to do is declare a string, let's name it *result*, cycle through the whole list of *books* (using the method *.size()*), and concatenate each book by a "\n" new line character into the *result* string itself.

At the end, we simply return *result*, that contains a line for each book.

Now we need 5 methods that are pretty similar between each other. They are called:

- *getBooksByAuthor*
- *getBooksByTitle*
- *getBooksByPublisher*
- *getBooksByISBN*
- *getBooksByYearOfPublication*

They all return a string, they all are *public*, and they all take as input a *string*, beside the last one that takes an integer.

The implementation is similar for all of them, let's do the first one together:

- we declare an array list of books, called *matchingBooks*, and instantiate it
- we create an *author*, based on the string parameter received
- we iterate through the set of books of our library
    - if a given book matches the picked author, we add it to our matching list
- at the end we need to return a string that formats all matching books in a nice representation; if only we had a method for this purpose…

Similarly you can implement the remaining four methods, the logic is awfully similar.

And that is all for the *Library* class!

server package

After modeling the core entities of our library application, it's time to move on to the server part that provides connection capabilities towards the network.

RequestId

Let's start by defining a simple enumeration that we will use for our message protocol. We need to distinguish whether the client is querying books' information based on author, title, publisher, isbn code or year of publication, and also if he simply wants to disconnect.

We will call our enumeration *RequestId* and it will contain the following values:

- *BYE*
- *AUTHOR*
- *TITLE*
- *PUBLISHER*
- *ISBN*
- *YEAR*

ClientHandler

The aim of our server application is to handle multiple client requests simultaneously.

Hence by definition we need to listen for incoming connections while handling communication towards the already established ones.

This is achieved by splitting the problem in two: one class will manage the new incoming connections (*Server*), and spawn instances of another class (*ClientHandler*) that focus on managing a single client at a time and provide meaningful responses.

The *ClientHandler* logic will hence need to work in parallel with respect to what is currently running on the *Server* side, or we would stop the management of incoming connections while replying to any query, and this would clearly be unacceptable.

For this reason let's define our *ClientHandler* class by extending the *Thread* java.lang class, that provides all we need to achieve this *parallel* execution.

Our class has the following attributes:

- a *private final* attribute named *socket* of type *Socket*: this is the object that abstracts for us all communication details towards the client
- a *private final* attribute named *library* of type *Library*: this is a reference to our books' library, needed to query the catalogue and provide responses to the client
- a *private* attribute named *writer* of type *PrintWriter*: we will use this object to write responses on the socket
- a *private* attribute named *reader* of type *BufferedReader*: we will use this object to read requests on the socket

The constructor receives and initializes the attributes *socket* and *library*.

Let's now define a method, called *handleRequest*, that receives a *String* named *request* and returns a *String* itself. The *private* access modifier will suffice.

This method needs to:

- extract the request id enumeration, in two steps:
    - extract an integer, named *requestId*, through the method *Integer.parseInt()*, passing as parameter the first character of the *request* parameter. We can extract the first character as follows: *request.substring(0, 1)*
    - convert the integer to enumeration with a bit of *reflection* wizardry:

      RequestId id = RequestId.values()[requestId];

- define a *String* variable, named *response*, initialized to the empty string content (“”)
- extract the actual request content, being an author or a title for example, again using the *substring* function:

  requestContent = request.substring(4, …)

  we start reading with *substring* from character 4, since we need to skip an integer of one digit followed by the delimiter “ - “ composed of three characters. Where shall the substring stop cutting our request?
- now we have the request *id* and the *requestContent*, let’s produce a *switch* statement on *id* that:
    - in case of *RequestId.AUTHOR*, sets the *response* variable to *library.getBooksByAuthor().* How can we pass the author to this function? Also, let’s not forget the *break* statement!
    - in case of *RequestId.TITLE,* sets the response variable to *library.getBooksByTitle()*, mimicking what already done in previous case
    - well, you got the drill, let’s cover also *PUBLISHER, YEAR* and *ISBN*
    - in case of the disconnect id *BYE*, the response shall be set to *"bye, see you next time!\n"*
    - and in case *default*, just to be safe, the response shall be set to *"Invalid message\n"*
- all request cases have been covered, time to return our *response* and conclude this function

Now that our *handleFunction* request is ready to be used, we just need to sort out the socket communication logic.

Let’s define a *public* function, named *run*, that returns nothing and takes no parameters. This is not yet another made-up function, this is the overridden function of the base class *Thread*, and represents the *parallel* logic that will kick-in as soon as we *start* our *ClientHandler.* As such, let’s not forget the *@Override* annotation.

This is what we need to implement in our *run* method:

- a *try* statement that:
    - creates a writer object, as follows:

      writer = new PrintWriter(socket.getOutputStream(), true);

- creates a reader object, as follows:

  reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));

- creates a variable, of type *String*, named *request*
- defines a *while* statement, that:
  - ★ checks in its condition that the read line is not null, as follows:

    while(request = reader.readLine is not null)

  - ★ prints out socket details, followed by the request, for you to have visibility (logging) of the communication details:

    System.out.println("[ClientHandler " + socket + "]");
    System.out.println(" [RX] " + request);

  - ★ defines a *response String* variable that shall be initialized to the result of invoking *handleRequest*. What we are doing here is producing the server' response the client's query
  - ★ once again, let's print out socket details, followed by the response to be transmitted:

    System.out.println("[ClientHandler " + socket + "]");
System.out.println(" [TX] " + response);

  - ★ let's transmit it on the socket, and don't forget to apply the "special" line at the end of the response, to mark that no more lines will follow:

    writer.println(response followed by the delimiter *"@@@")*

  - ★ if the *response* we just sent out contains the string "*bye"*, it means that our client will soon disconnect, and we need to terminate the *while* loop, otherwise infinite:

    System.out.println("[ClientHandler " + socket + "]");
    System.out.println(" Socket closing");
    break;

- our loop is over, so we close our socket since client is ready to disconnect and will no longer produce queries:

  reader.close();
  writer.close();
  socket.close();

- Just some housekeeping left, let's finish up our *try-catch* statement by catching an *IOException,* and printing out the exception content if something bad happens.

And that is all for *ClientHandler*, time to move to *Server* class.

Server

Fairly simple class to implement, since all the logic is provided by the *java* framework.

Our class has three attributes:

- a *private final* reference to *Library*, named *library*
- a *private* attribute named *serverSocket*, of type *ServerSocket*; please note how this is a different type with respect to the socket in use by the *ClientHandler*!
- a *private final* attribute named *port* of type *int*, initialized to **1234**. This is the port number that we will listen on, and clearly it is purely arbitrary.

The constructor simply receives and initializes the *library* reference.

We need to define two methods, *start* and *stop*.

The *stop* method is trivial, let's get it out first. It takes no parameters, returns *void*, and may throw an *IOException*. All it does is invoking *close()* on the *serverSocket* attribute.

The *start* method is slightly more complicated. Once again it takes no parameters, returns *void*, and may throw an *IOException*.

It needs to:

- create the *serverSocket*, by passing the *port* parameter
- define an infinite *while* statement, that:
    - defines a *Socket s* variable; this is the exact type used in *ClientHandler*, is it a coincidence?
    - sets this variable to the return value of *serverSocket.accept*; this functions halts the execution waiting for an incoming connection, and returns on successful establishment of communication
    - time to define a *ClientHandler* variable, named simply *handler*, by passing our newly created socket and the library reference
    - let's invoke the *start* method on the variable *handler*, that kicks the *thread* nature of it, and invokes the underlying *run* function for us

That is all, simple enough right? We wait, we establish the communication, we kick a dedicated handler *thread*, and we go back listening for incoming connections.
Divide-et-impera at its best!

Main

Time to define our *main* method, to speed things up here below you can find the implementation. There's nothing fancy, it just implements few searches on the catalogue, just to make sure everything is in place, before launching our server:

```
Library library = new Library();

String results;
```

```
results = library.getBooksByAuthor("Gaiman");
System.out.println("----------------------------------");
System.out.println("Looking for books written by Gaiman\n");
System.out.print(results);
System.out.println("----------------------------------\n");

results = library.getBooksByTitle("Android");
System.out.println("----------------------------------");
System.out.println("Looking for books containing the word Android in the title\n");
System.out.print(results);
System.out.println("----------------------------------\n");

results = library.getBooksByPublisher("O'Reilly");
System.out.println("----------------------------------");
System.out.println("Looking for books published by O'Reilly\n");
System.out.print(results);
System.out.println("----------------------------------\n");

results = library.getBooksByYearOfPublication(1986);
System.out.println("----------------------------------");
System.out.println("Looking for books published in year 1986\n");
System.out.print(results);
System.out.println("----------------------------------\n");

results = library.getBooksByISBN("140431179");

System.out.println("----------------------------------");
System.out.println("Looking for book with a given ISBN code\n");
System.out.print(results);
System.out.println("----------------------------------\n");

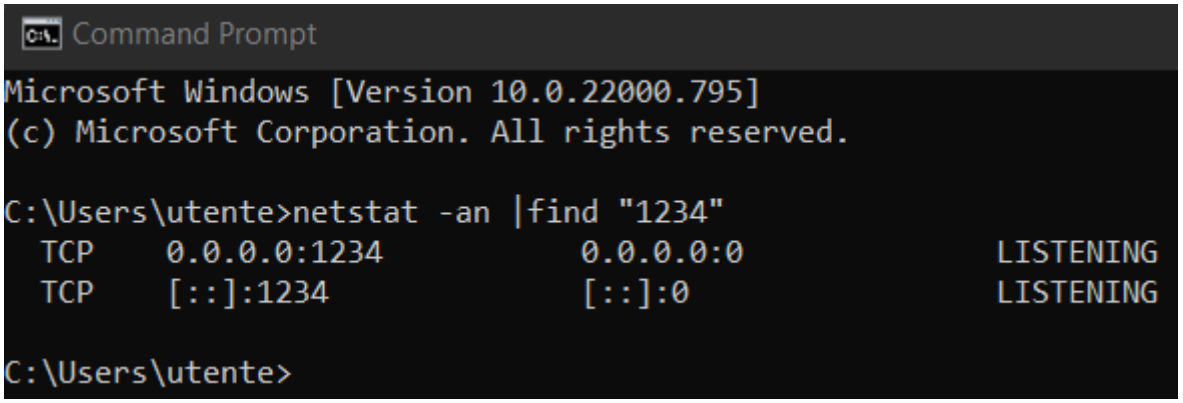
Server server = new Server(library);
server.start();
```

Now run the *libraryServer* project, and you will see that it never concludes (the server needs to stay alive forever, until manually shut down).

Just to make sure that the connection is open, on a *Windows* system open up a *Command Prompt* and type:

```
netstat -an | find “1234”
```

You should see the following output:



That, in human-readable format, means that our application listens on connections coming from any *IP address* on port **1234**.

On *MacOS* or *Linux*, you can type *netstat -an | grep 1234* on a *terminal* instead.

Server is done, time to move the client side, that will be really simpler.

Library Client

Let's start by creating a java application named *libraryClient*, composed of a single package named *client*.

client package

RequestId

Nothing new here, just copy what already implemented on the server side. After all, clients and server need to share the same message protocol!

Client

This class is composed of three attributes, all *private*, in particular:

- an attribute named *socket*, of type *Socket*. Note how this is the same object type in use by the ClientHandler! Java designers were clever enough to figure out that once a connection is established, being client or server is pretty much equivalent, hence the harmonized symmetry of objects.
- an attribute named *writer*, of type *PrintWriter*
- an attribute named *reader*, of type *BufferedReader*

No need for a constructor, the default one will do.

Let's now define two simple methods, *start* and *stop*.

The *start* method receives as parameters a *port* (*int* type), and an *address* (*String* type). It creates all attributes, as follows:

```
socket = new Socket(ip, port);
writer = new PrintWriter(socket.getOutputStream(), true);
reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

It may throw an *IOException*.

Nothing else to be done for this method, what return type would you choose?

The *stop* method is mind-blowing, it takes no parameters, it has no return value, it may throw an *IOException*, and invoke close on *reader*, *writer* and *socket*.

We need one more method, called *transmitAndReceive*. This method sends a request to the server counterpart, and waits for its response.

It shall take a parameter of type *String*, named *request*, and shall return the response, being it a *String* as well. It may throw an *IOException*.

It shall implement the following logic:

- print out the socket details and the request, for your readability:

  ```
  System.out.println("[Client " + socket + "]");
  System.out.println("       [TX] " + msg);
  ```

- transmit the *request*, using *writer* and the *println* function, seen already in previous chapters
- defines two *String* variables, initialized to empty strings, named *response* and *line*
- define an infinite *do-while* statement, that:
  - sets *line* to the result of a single *reader.readLine* invoke
  - if *line* is equal to “@@@”, invoke *break* to interrupt this infinite cycle
  - append *line* to *response*, followed by a *“\n”*. The server formatted a “\n” for every matching book, that is true, but the function *readLine* digested it for us!
- print out the socket details and the response:

  ```
  System.out.println("[Client " + socket + "]");
  System.out.println("       [RX] " + response);
  ```

- return *response*

We now have means to start, stop and transmit through our clients, let’s test it out in our main!

Main

Nothing fancy, our main function shall test our set of applications by:

- creating a client instance
- start it with the *start* method, passing for example *address* “127.0.0.1” and *port* ***1234*** when running on the same host of the server
- try out some queries, here are few examples:

c1.transmitAndReceive(RequestId.AUTHOR.ordinal() + " - Lewis Carroll");
c1.transmitAndReceive(RequestId.TITLE.ordinal() + " - on the road");

- when you are satisfied with the catalogue's queries, don't forget to disconnect properly to avoid leaving a *server ClientHandler* hanging for nothing:

  c1.transmitAndReceive(RequestId.BYE.ordinal() + " - bye bye");
  c1.stop();

That's it, your first client-server application in all its glory, well done!
